

گزارش کار تمرین کامپیوتری ششم درس سیگنال و سیستم ها

امیرحسین صباحی

810101556

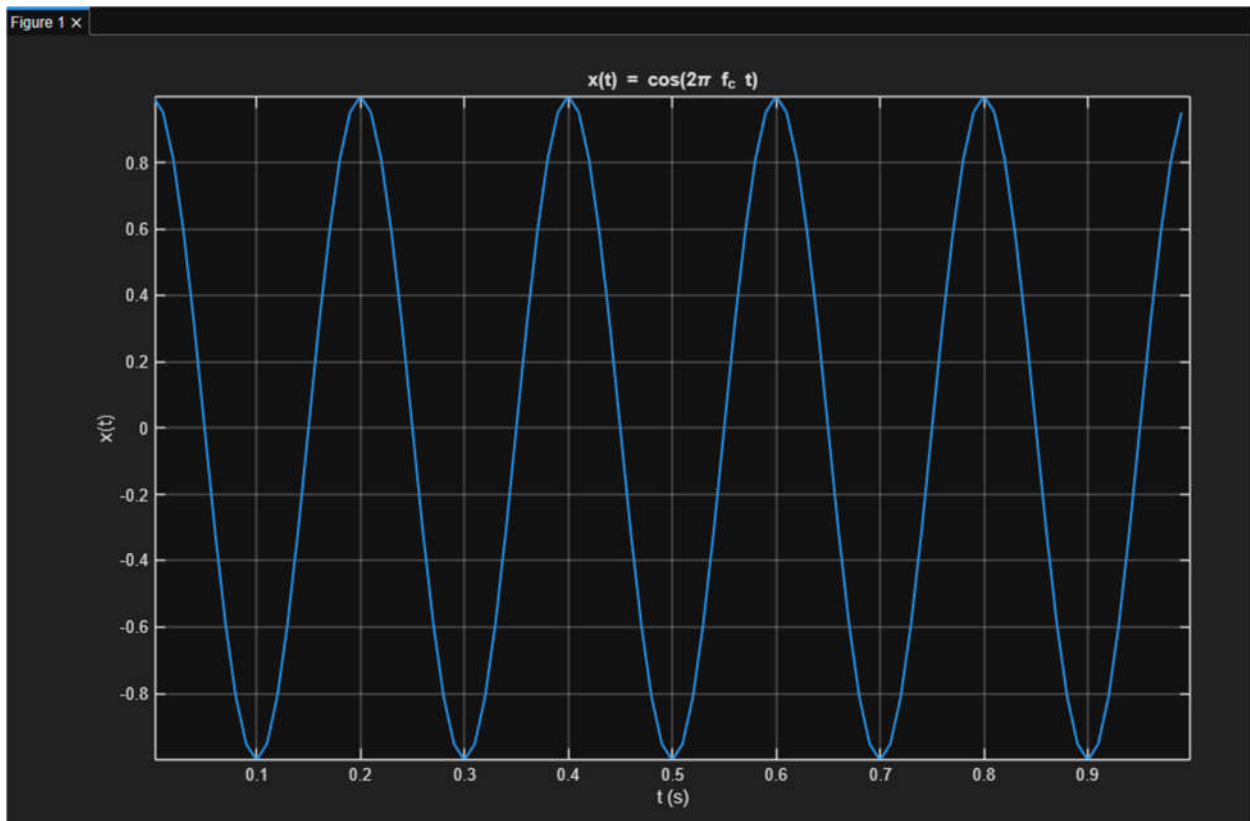
بخش اول

(1.1)

در فایل run1.m

سیگنال ارسالی رادار به صورت $x(t) = \cos(2\pi f_c t)$ مدل شد و برای شبیه سازی آن در محیط متلب بازه زمانی از ۰ تا ۱ ثانیه با نرخ نمونه برداری $f_s = 100\text{Hz}$ در نظر گرفته شد و با استفاده از $t = 0:Ts:1-Ts$ و $Ts = 1/f_s$ مقادیر گسسته سیگنال محاسبه و رسم گردید با انتخاب $f_c = 5\text{Hz}$ شکل موج سینوسی با پنج دوره در طول یک ثانیه مشاهده شد که نشان می دهد نمونه برداری انجام شده برای نمایش مناسب سیگنال کافی است و سیگنال تولید شده مبنای مراحل بعدی تمرین برای مقایسه با سیگنال دریافتی و تحلیل فرکانسی قرار گرفت.

```
term/Signalca6.m
1 %1.1
2 clc; clear; close all;
3
4 fc = 5;
5 fs = 100;
6 tstart = 0;
7 tend = 1;
8
9 Ts = 1/fs;
10 t = tstart : Ts : (tend - Ts);
11
12 x = cos(2*pi*fc*t);
13
14 figure;
15 plot(t, x, 'LineWidth', 1.5);
16 grid on;
17 xlabel('t (s)');
18 ylabel('x(t)');
19 title('x(t) = cos(2\pi f_c t)');
20
```



1.2

سیگنال دریافتی به صورت $y(t) = \alpha \cos(2\pi(f_c + f_d)(t - t_d))$ در نظر گرفته شد که در آن تغییر فرکانس ناشی از اثر داپلر و تاخیر زمانی ناشی از فاصله هدف است ابتدا با استفاده از رابطه $f_d = \beta V$ و مقدارهای داده شده $\beta = 0.3$ و $V = 180 \text{ km/h}$ مقدار f_d محاسبه شد و سپس با فرض $t_d = pR$ و $p = 2/c$ با $c = 3 \times 10^5 \text{ km/s}$ و $R = 250 \text{ km}$ تاخیر زمانی به دست آمد در ادامه با قرار دادن $\alpha = 0.5$ و $f_c = 5 \text{ Hz}$ و استفاده از همان بردار زمان و نرخ نمونه برداری تمرین قبل سیگنال دریافتی تولید و رسم شد و مشاهده گردید که نسبت به سیگنال ارسالی هم تغییر فاز دارد و هم به علت تغییر فرکانس تعداد نوسان ها در بازه یک ثانیه متفاوت می شود که این تغییرات پایه استخراج سرعت و فاصله در بخش های بعدی است.

```

%1.2
fc = 5;
fs = 100;
tstart = 0;
tend = 1;

alpha = 0.5;
beta = 0.3;
V = 180;
R = 250;

c0 = 3e5;
rho = 2/c0;

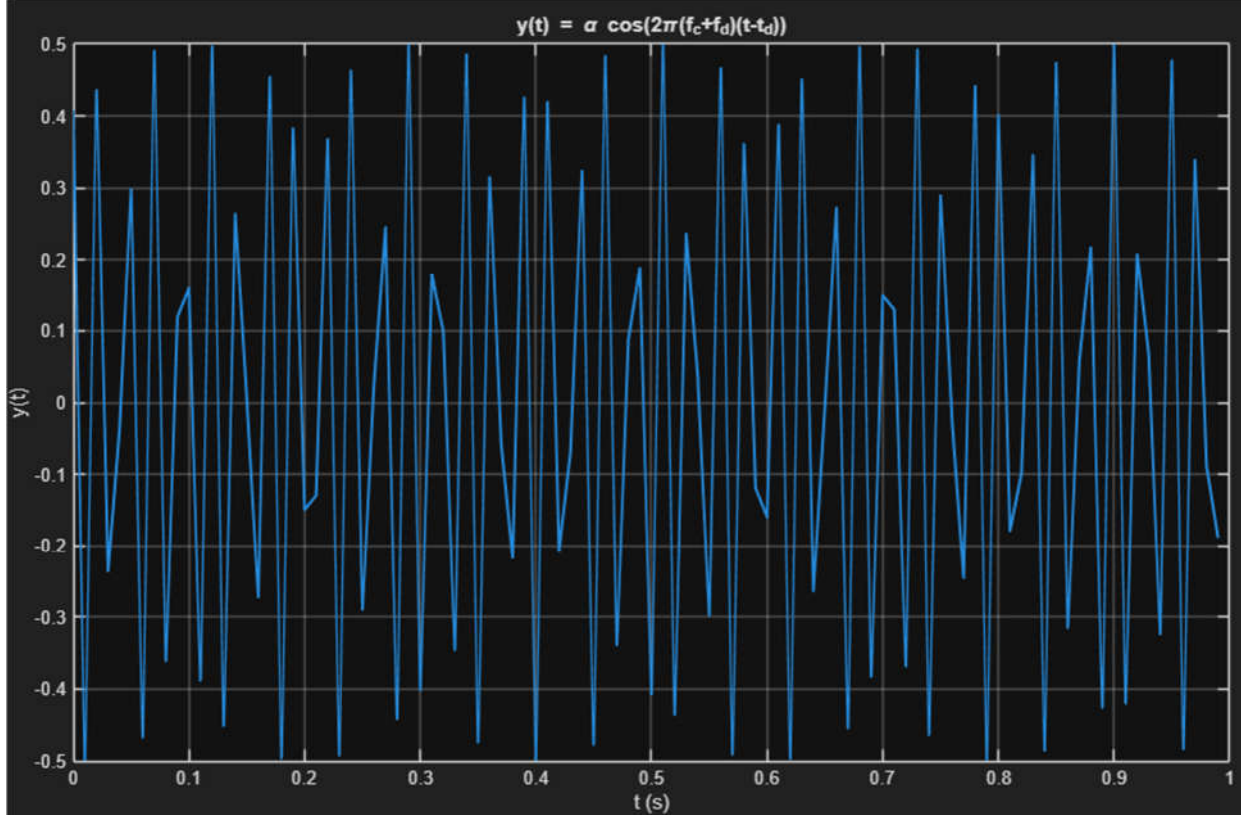
fd = beta*V;
td = rho*R;

Ts = 1/fs;
t = tstart:Ts:(tend-Ts);

y = alpha*cos(2*pi*(fc+fd).*(t-td));

figure;
plot(t, y, 'LineWidth', 1.5);
grid on;
xlabel('t (s)');
ylabel('y(t)');
title('y(t) = \alpha cos(2\pi(f_c+f_d)(t-t_d))');

```



1.3

در این بخش هدف این بود که تنها با داشتن سیگنال دریافتی بتوان سرعت و فاصله را به صورت خودکار تخمین زد ابتدا سیگنال دریافتی به حوزه فرکانس منتقل شد و با محاسبه FFT و جابه جایی طیف با `fftshift` محور فرکانس ساخته شد سپس با بررسی قدر مطلق طیف در نیمه مثبت فرکانس پیک غالب انتخاب گردید و این فرکانس به عنوان f_{new} در مدل $y(t) = \alpha \cos(2\pi f_{new} t + \phi_{new})$ در نظر گرفته شد همزمان فاز متناظر همان بین فرکانسی از `angle` استخراج و به عنوان ϕ_{new} ثبت شد تا اطلاعات لازم برای بازسازی تاخیر زمانی از روی فاز فراهم شود.

پس از به دست آوردن f_{new} و ϕ_{new} ابتدا مولفه داپلر از رابطه $f_d = f_{new} - f_c$ محاسبه شد و با استفاده از $f_d = \beta V$ مقدار سرعت به صورت $V = (f_{new} - f_c) / \beta$ به دست آمد سپس از رابطه فاز در سیگنال کسینوسی نتیجه شد که $\phi_{new} = -2\pi f_{new} t_d$ و بنابراین تاخیر زمانی با $t_d = \text{mod}(-\phi_{new} / (2\pi f_{new}), 1/f_{new})$ محاسبه گردید و در نهایت فاصله هدف از $R = t_d / \rho$ استخراج شد با این روش کل فرآیند تخمین بر پایه دو ویژگی اصلی طیف یعنی محل قله فرکانسی برای سرعت و فاز همان قله برای فاصله انجام شد و خروجی ها به صورت خودکار از روی سیگنال دریافتی محاسبه گردید.

```
%1.3
fc = 5;
fs = 100;
T = 1;

alpha = 0.5;
beta = 0.3;

V0 = 180;
R0 = 250;

c0 = 3e5;
rho = 2/c0;

t = 0:1/fs:(T-1/fs);
N = numel(t);

fd0 = beta*V0;
td0 = rho*R0;

y = alpha*cos(2*pi*(fc+fd0).*(t-td0));

Y = fftshift(fft(y));
f = (-N/2:N/2-1)*(fs/N);

pos = find(f > 0);
[~,ii] = max(abs(Y(pos)));
k = pos(ii);
fpk = f(k);

cand = [fpk, fs-fpk];
if abs(cand(1)-cand(2)) < 1e-9
    cand = cand(1);
end

best.err = inf;
```

```

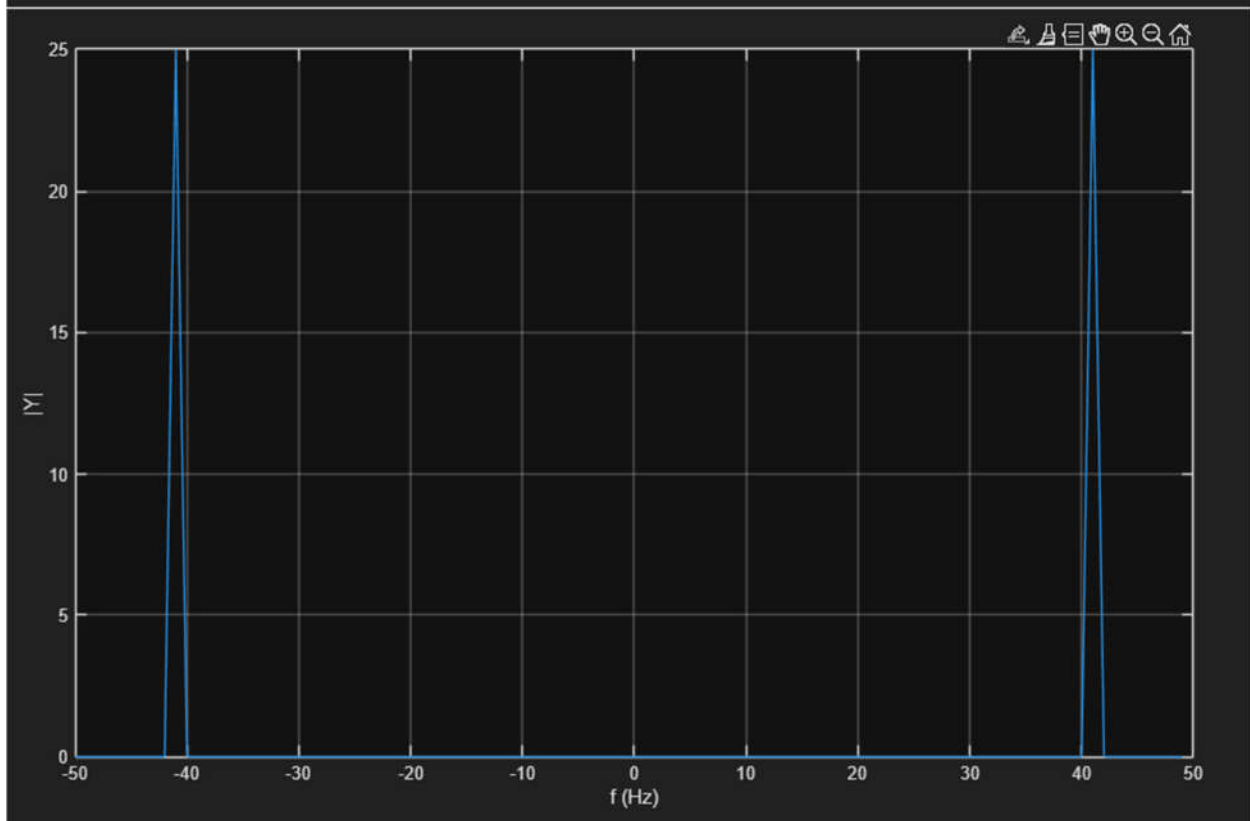
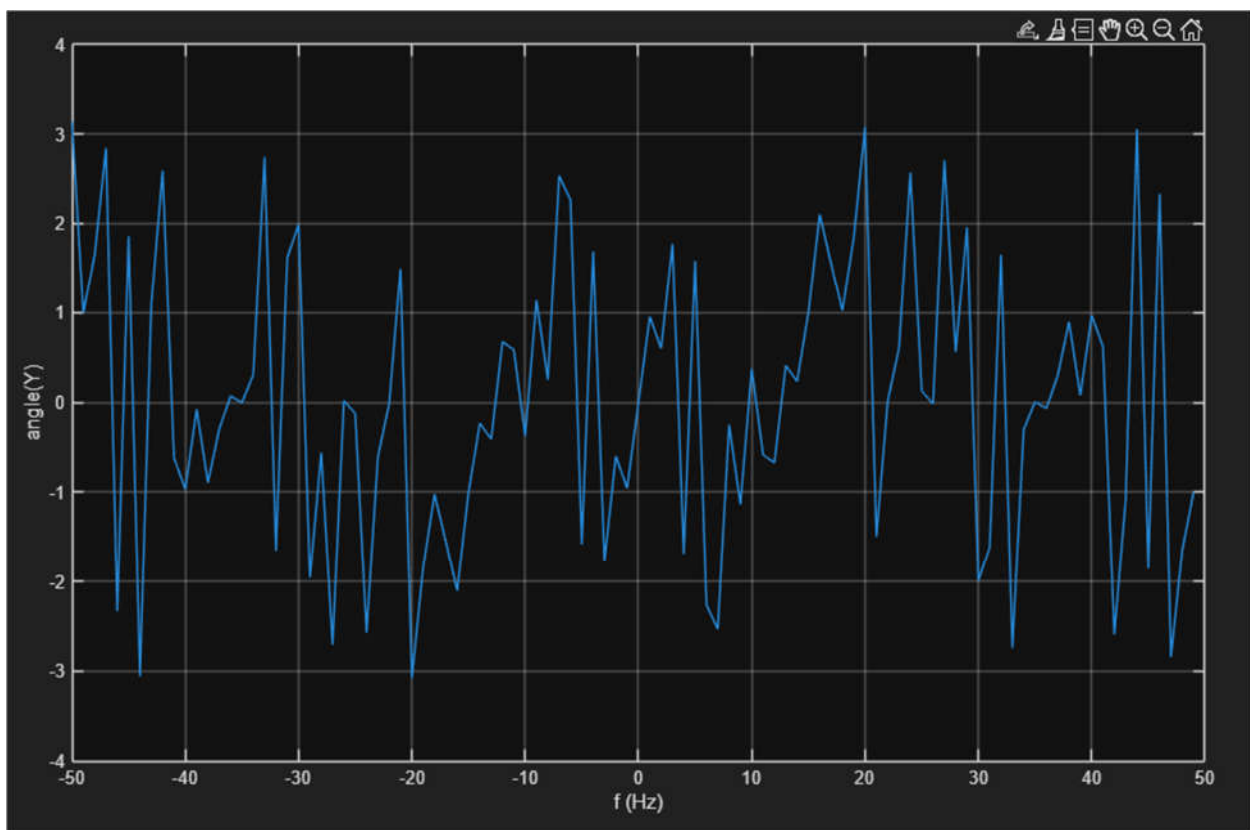
87     for f0 = cand
88         C = [cos(2*pi*f0*t(:)) sin(2*pi*f0*t(:))];
89         p = C\y(:);
90         a = p(1); b = p(2);
91
92         A = hypot(a,b);
93         phi = atan2(-b,a);
94
95         td = mod(-phi/(2*pi*f0), 1/f0);
96
97         yh = A*cos(2*pi*f0*(t-td));
98         e = mean((y - yh).^2);
99
100        if e < best.err
101            best.err = e;
102            best.f = f0;
103            best.phi = phi;
104            best.td = td;
105            best.A = A;
106        end
107    end
108
109    f_new = best.f;
110    phi_new = best.phi;
111    t_d = best.td;
112
113    V_hat = (f_new - fc)/beta;
114    R_hat = t_d/rho;
115
116    fprintf('f_new = %.6f Hz\n', f_new);
117    fprintf('phi_new = %.6f rad\n', phi_new);
118    fprintf('t_d = %.9f s\n', t_d);
119    fprintf('V_hat = %.6f km/h\n', V_hat);
120    fprintf('R_hat = %.6f km\n', R_hat);
121
122    figure; plot(f, abs(Y)); grid on; xlabel('f (Hz)'); ylabel('|Y|');
123    figure; plot(f, angle(Y)); grid on; xlabel('f (Hz)'); ylabel('angle(Y)');
124

```

```

f_new      = 59.000000 Hz
phi_new    = -0.617847 rad
t_d        = 0.001666667 s
V_hat      = 180.000000 km/h
R_hat      = 250.000000 km

```



(1.4

برای بررسی تاثیر نویز ابتدا به سیگنال دریافتی نویز سفید گاوسی با توان کنترل شده اضافه شد و سپس همان روند بخش ۱ ۳ شامل محاسبه FFT استخراج فرکانس غالب و فاز متناظر و در ادامه محاسبه f_d و t_d و تبدیل آن ها به V و R تکرار گردید قدرت نویز در چند مرحله افزایش داده شد و با مقایسه خطای نسبی خروجی ها مشخص شد تا زمانی که قله فرکانسی در طیف قابل تشخیص باشد تخمین سرعت پایدارتر باقی می ماند اما تخمین فاصله حساسیت بیشتری نشان می دهد زیرا فاصله از روی تاخیر زمانی و در عمل از روی فاز استخراج می شود و فاز در حضور نویز سریع تر دچار نوسان و پرش می گردد بنابراین در سطوح نویز بالاتر معمولاً ابتدا دقت فاصله کاهش پیدا می کند و سپس در صورت محو شدن قله ها دقت سرعت نیز افت می کند.

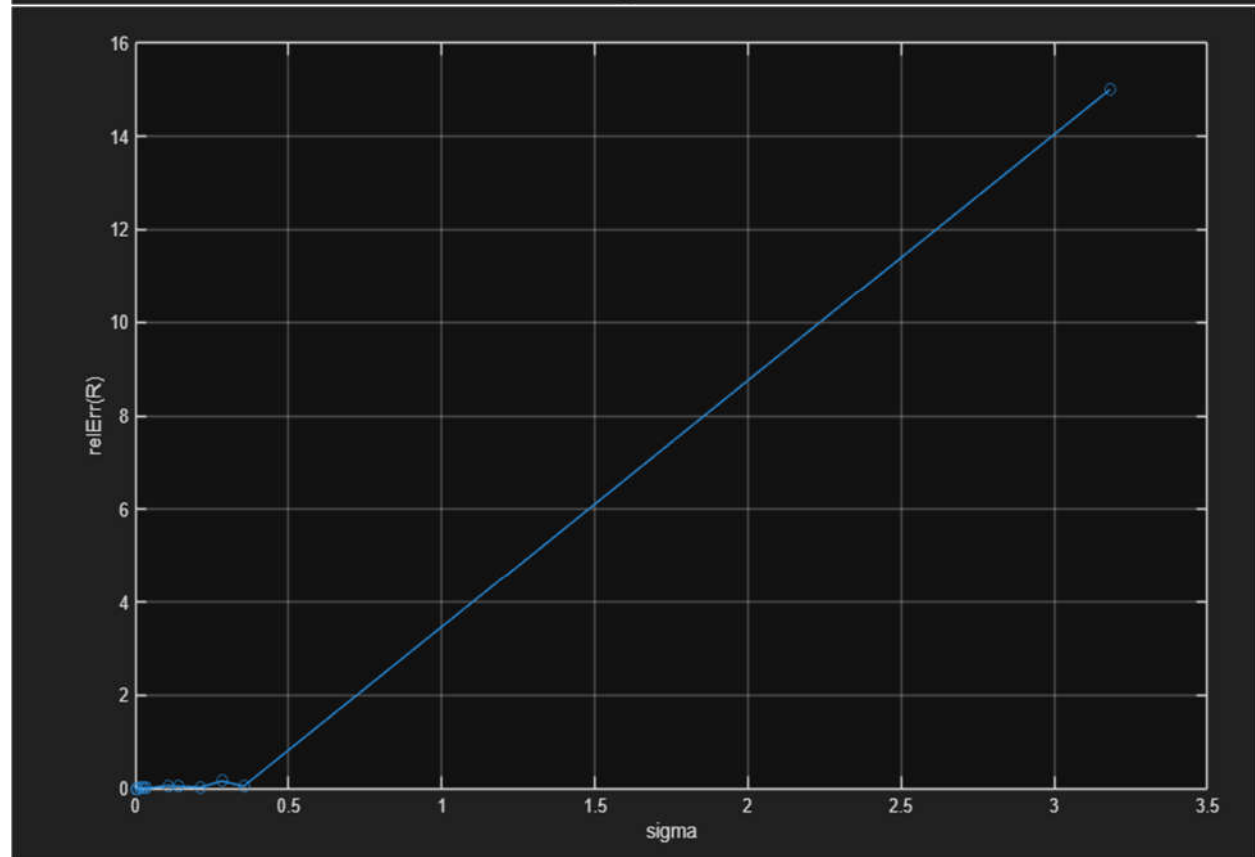
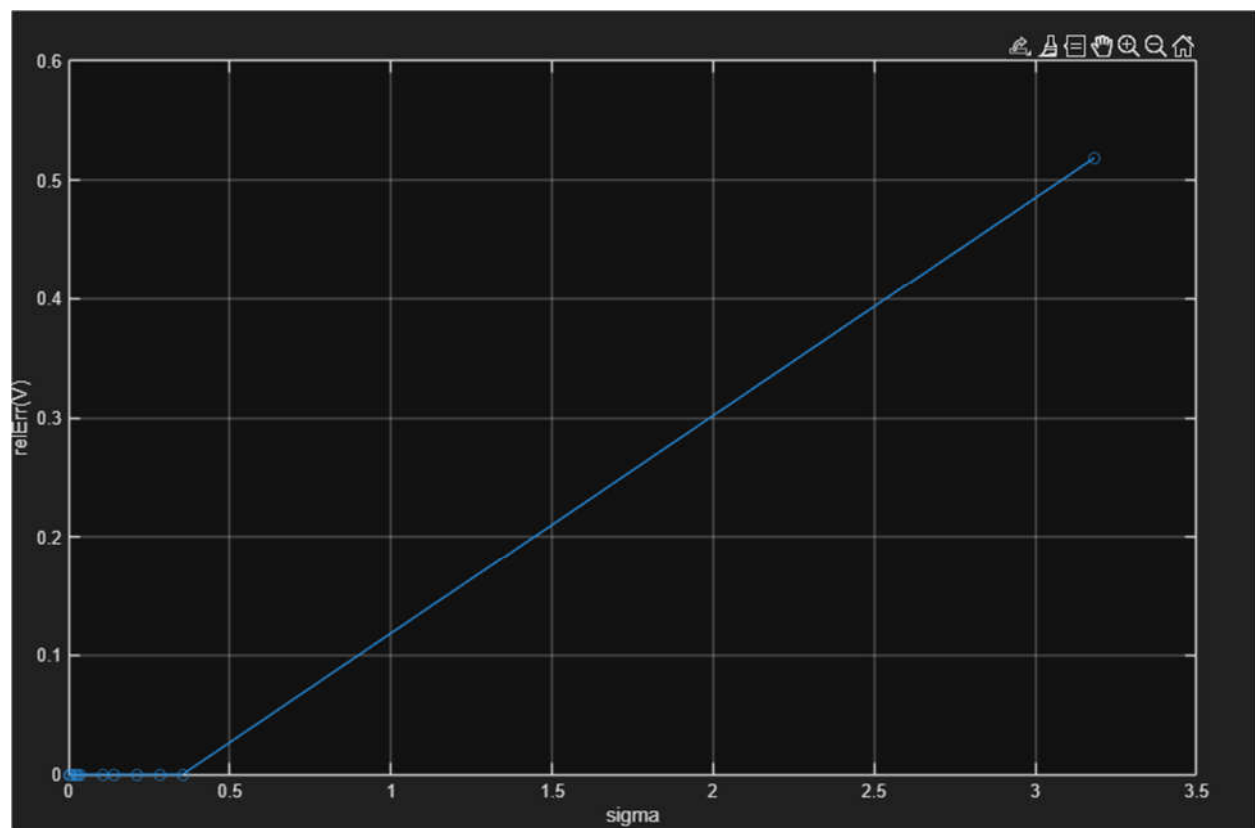
```
125 %1.4
126 rng(1);
127
128 fc = 5;
129 fs = 120;
130 T = 1;
131
132 alpha = 0.5;
133 beta = 0.3;
134
135 vtrue = 180;
136 Rtrue = 250;
137
138 c0 = 3e5;
139 rho = 2/c0;
140
141 t = 0:1/fs:(T-1/fs);
142
143 fd = beta*vtrue;
144 td = rho*Rtrue;
145
146 y = alpha*cos(2*pi*(fc+fd).*(t-td));
147
148 sigRms = sqrt(mean(y.^2));
149 sigmaList = sigRms * [0 0.01 0.02 0.04 0.06 0.08 0.10 0.30 0.40 0.60 0.80 1.00 9.00];
150
151 out = zeros(numel(sigmaList), 6);
152
153 for i = 1:numel(sigmaList)
154     sigma = sigmaList(i);
155
156     yN = y + sigma*randn(size(y));
157
158     [Vhat, Rhat, fnew, tdHat] = estVR_noalias(yN, t, fs, fc, beta, rho);
159
160     eV = abs(Vhat - vtrue)/abs(vtrue);
161     eR = abs(Rhat - Rtrue)/abs(Rtrue);
162
163     out(i,:) = [sigma, Vhat, Rhat, eV, eR, fnew];
164 end
```

```

5
6 disp(array2table(out, 'VariableNames', {'sigma','V_hat','R_hat','relErrV','relErrR','f_new'}));
7
8 figure; plot(out(:,1), out(:,4), 'o-'); grid on;
9 xlabel('sigma'); ylabel('relErr(V)');
10
11 figure; plot(out(:,1), out(:,5), 'o-'); grid on;
12 xlabel('sigma'); ylabel('relErr(R)');
13
14 function [Vhat, Rhat, fnew, td] = estVR_noalias(y, t, fs, fc, beta, rho)
15     N = numel(t);
16
17     Y = fftshift(fft(y));
18     f = (-N/2:N/2-1)*(fs/N);
19
20     ip = find(f > 0);
21     [~,im] = max(abs(Y(ip)));
22     k = ip(im);
23
24     fnew = f(k);
25     if fnew < 1e-9
26         fnew = 1e-9;
27     end
28
29     C = [cos(2*pi*fnew*t(:)) sin(2*pi*fnew*t(:))];
30     p = C\y(:);
31
32     a = p(1);
33     b = p(2);
34
35     A = hypot(a,b);
36     phi = atan2(-b,a);
37
38     td = mod(-phi/(2*pi*fnew), 1/fnew);
39
40     Vhat = (fnew - fc)/beta;
41     Rhat = td/rho;
42 end

```

sigma	V_hat	R_hat	relErrV	relErrR	f_new
0	180	250	0	1.0687e-14	59
0.0035355	180	249.66	0	0.0013642	59
0.0070711	180	250.12	0	0.00047768	59
0.014142	180	247.46	0	0.010144	59
0.021213	180	253.02	0	0.012096	59
0.028284	180	245.24	0	0.019049	59
0.035355	180	250.81	0	0.0032212	59
0.10607	180	265.17	0	0.060699	59
0.14142	180	261.8	0	0.047195	59
0.21213	180	243.72	0	0.025108	59
0.28284	180	289.74	0	0.15894	59
0.35355	180	261.61	0	0.046442	59
3.182	86.667	4002.2	0.51852	15.009	31



(1.5)

در این بخش حالت دو هدف ایده آل بدون نویز بررسی شد و سیگنال دریافتی به صورت جمع دو اکو مدل گردید یعنی $y(t) = \alpha_1 \cos(2\pi(fc+fd_1)(t-td_1)) + \alpha_2 \cos(2\pi(fc+fd_2)(t-td_2))$ که در آن برای هر هدف به طور جداگانه $fd = \beta V$ و $td = \rho R$ محاسبه می شود با قرار دادن مقادیر داده شده برای دو جسم شامل R_1 و V_1 و α_1 و نیز R_2 و V_2 و α_2 و استفاده از همان نرخ نمونه برداری و بازه زمانی بخش های قبل سیگنال ترکیبی ساخته و رسم شد و مشاهده گردید که شکل موج نسبت به حالت تک هدف پیچیده تر است چون دو مولفه با فرکانس ها و فازهای متفاوت همزمان حضور دارند و همین همپوشانی باعث تغییرات دامنه و الگوی نوسان متفاوت در زمان می شود که مبنای بخش های بعدی برای جداسازی دو هدف از روی طیف و استخراج سرعت و فاصله هر کدام است.

```
%1.5
fc = 5;
fs = 100;
t = 0:1/fs:(1-1/fs);

beta = 0.3;

R1 = 250; V1 = 180; a1 = 0.5;
R2 = 200; V2 = 216; a2 = 0.6;

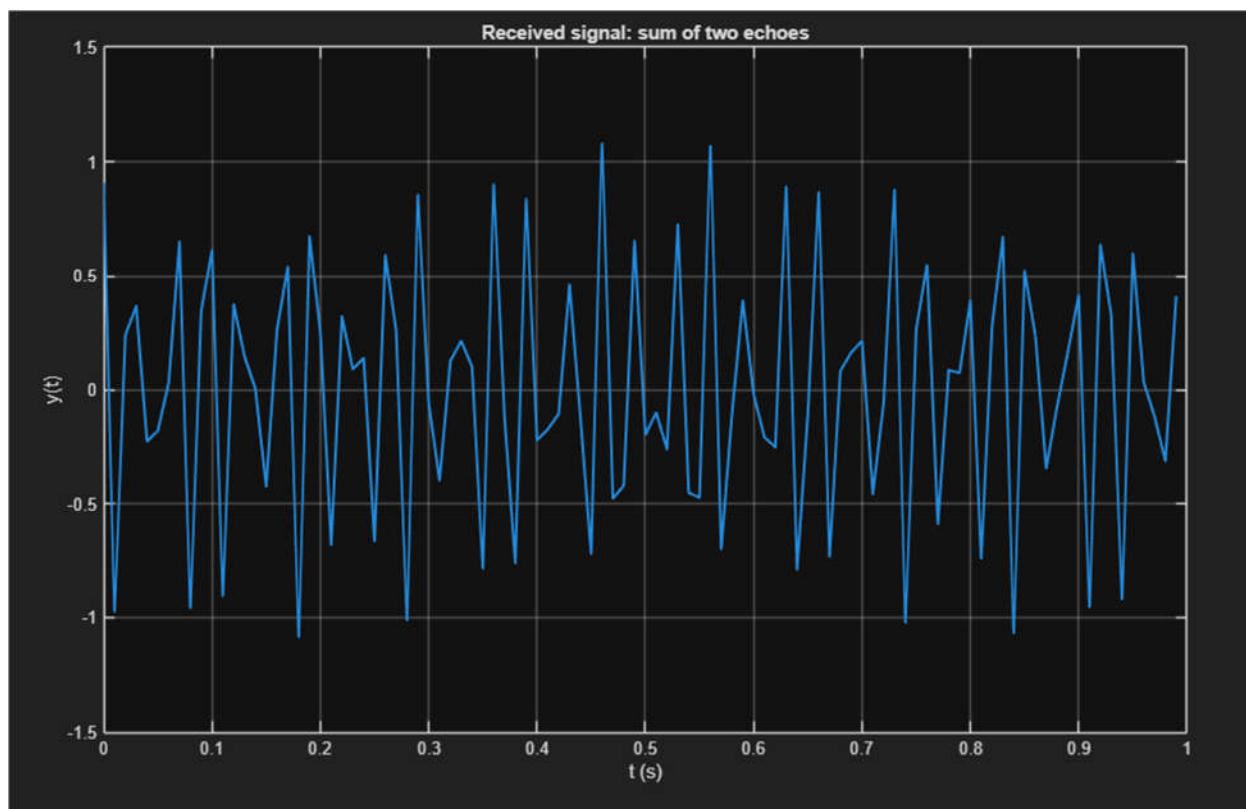
c0 = 3e5;
rho = 2/c0;

fd1 = beta*V1;
fd2 = beta*V2;

td1 = rho*R1;
td2 = rho*R2;

y = a1*cos(2*pi*(fc+fd1).*(t-td1)) + a2*cos(2*pi*(fc+fd2).*(t-td2));

figure;
plot(t, y, 'LineWidth', 1.5);
grid on;
xlabel('t (s)');
ylabel('y(t)');
title('Received signal: sum of two echoes');
```



(1.6

در این بخش مسئله به صورت معکوس بررسی شد یعنی فرض شد تنها سیگنال دریافتی دو هدفه در اختیار است و باید از روی آن سرعت و فاصله هر دو جسم به دست آید برای این کار ابتدا سیگنال به حوزه فرکانس منتقل شد و با محاسبه FFT در نیمه مثبت طیف دو قله اصلی شناسایی گردید تا دو فرکانس غالب مربوط به دو هدف استخراج شود سپس برای کاهش اثر تداخل دو مولفه روی فاز به جای اتکا صرف به فاز FFT از برازش کمترین مربعات در حوزه زمان استفاده شد و ضرایب سینوس و کسینوس هر فرکانس به دست آمد تا فاز هر مولفه محاسبه و از روی آن تاخیر زمانی هر هدف استخراج گردد بعد از آن با استفاده از $f_d = f_{\text{new}} - f_c$ و $V = f_d / \beta$ سرعت هر هدف محاسبه شد و با داشتن td فاصله نیز از $R = td / \rho$ به دست آمد و در نهایت نتایج با پارامترهای تولید شده در بخش ۱ مقایسه شد تا صحت روش سنجیده شود همچنین اگر فرکانس واقعی از نایکوئیست عبور کند پدیده آلیاسینگ می تواند باعث ابهام در تشخیص فرکانس و در نتیجه خطا در سرعت شود و برای کاهش این مشکل افزایش نرخ نمونه برداری یا افزایش طول مشاهده و بهبود تفکیک فرکانسی می تواند مفید باشد.

```

%1.6
rng(1);

fc = 5;
fs = 100;
T = 1;

beta = 0.3;

c0 = 3e5;
rho = 2/c0;

t = 0:1/fs:(T-1/fs);

R1 = 250; V1 = 180; A1 = 0.5;
R2 = 200; V2 = 216; A2 = 0.6;

fd1 = beta*V1; td1 = rho*R1;
fd2 = beta*V2; td2 = rho*R2;

y = A1*cos(2*pi*(fc+fd1).*(t-td1)) + A2*cos(2*pi*(fc+fd2).*(t-td2));

Nfft = 16384;

Y = fft(y, Nfft);
f = (0:Nfft/2)*fs/Nfft;
mag = abs(Y(1:Nfft/2+1));
mag(1) = 0;

pk = find(mag(2:end-1) > mag(1:end-2) & mag(2:end-1) >= mag(3:end)) + 1;
if numel(pk) < 2
    [~,ix] = sort(mag,'descend');
    pk = ix(1:2);
end

[~,ord] = sort(mag(pk),'descend');
pk = pk(ord);

k1 = pk(1);
k2 = pk(find(abs(pk - k1) >= 10, 1, 'first'));
if isempty(k2)
    k2 = pk(2);
end

df = fs/Nfft;

f1obs = quad_refine(f, mag, k1, df);
f2obs = quad_refine(f, mag, k2, df);

C = [cos(2*pi*f1obs*t(:)) sin(2*pi*f1obs*t(:)) cos(2*pi*f2obs*t(:)) sin(2*pi*f2obs*t(:))];
p = C\y(:);

a1 = p(1); b1 = p(2);
a2 = p(3); b2 = p(4);

phi1 = atan2(-b1,a1);
phi2 = atan2(-b2,a2);

```

```

[f1, td1h, V1h, R1h] = decode_one(f1obs, phi1, fs, fc, beta, rho);
[f2, td2h, V2h, R2h] = decode_one(f2obs, phi2, fs, fc, beta, rho);

if V2h > V1h
    [f1,f2] = deal(f2,f1);
    [td1h,td2h] = deal(td2h,td1h);
    [V1h,V2h] = deal(V2h,V1h);
    [R1h,R2h] = deal(R2h,R1h);
end

fprintf('Target 1: f=%.4f Hz, td=%.6f s, V=%.3f, R=%.3f\n', f1, td1h, V1h, R1h);
fprintf('Target 2: f=%.4f Hz, td=%.6f s, V=%.3f, R=%.3f\n', f2, td2h, V2h, R2h);

figure; plot(f, mag, 'Linewidth', 1.1); grid on;
xlabel('f (Hz)'); ylabel('|Y|');

figure; plot(t, y, 'Linewidth', 1.1); grid on;
xlabel('t (s)'); ylabel('y(t)');

function f0 = quad_refine(f, mag, k, df)
    if k <= 1 || k >= numel(mag)
        f0 = f(k);
        return;
    end
    m1 = mag(k-1); m0 = mag(k); m2 = mag(k+1);
    den = (m1 - 2*m0 + m2);
    if abs(den) < 1e-12
        d = 0;
    else
        d = 0.5*(m1 - m2)/den;
    end
    f0 = f(k) + d*df;
end

function [ftrue, td, V, R] = decode_one(fobs, phi, fs, fc, beta, rho)
    fA = fobs;
    tdA = mod(-phi/(2*pi*fA), 1/fA);
    VA = (fA - fc)/beta;
    RA = tdA/rho;

    fB = fs - fobs;
    tdB = mod(+phi/(2*pi*fB), 1/fB);
    VB = (fB - fc)/beta;
    RB = tdB/rho;

    if (RB < RA) && (VB > 0)
        ftrue = fB; td = tdB; V = VB; R = RB;
    else
        ftrue = fA; td = tdA; V = VA; R = RA;
    end
end

```

```

Target 1: f=69.8013 Hz, td=0.001345 s, V=216.004, R=201.813
Target 2: f=58.9683 Hz, td=0.001397 s, V=179.894, R=209.496
>> |

```

(1.7)

اگر سرعت دو جسم برابر باشد آنگاه فرکانس داپلر آن ها هم برابر می شود یعنی $f_{d1}=f_{d2}$ و بنابراین هر دو اکو در گیرنده روی یک فرکانس یکسان ظاهر می شوند و جمع دو سیگنال هم به یک سیگنال تک فرکانس معادل با یک دامنه و یک فاز جدید تبدیل می شود در نتیجه از روی یک مشاهده زمانی فقط یک f_{new} و یک ϕ_{new} معنادار استخراج می شود و به طور یکتا نمی توان دو تاخیر t_{d1} و t_{d2} در نتیجه دو فاصله متفاوت را جداگانه به دست آورد مگر اینکه اطلاعات اضافه مثل پهنای باند بیشتر یا چند مشاهده در زمان های مختلف داشته باشیم همچنین برای اینکه دو هدف از روی طیف از هم تفکیک شوند باید اختلاف فرکانس دو مولفه از رزولوشن فرکانسی بزرگ تر باشد یعنی $\Delta f \geq \delta f$ و چون $\delta f = 1/T$ و $f_d = \beta V$ داریم $\Delta V_{min} = \delta f / \beta = 1/(\beta T)$ و با فرض های این تمرین که $T=1s$ و $\beta=0.3$ است حداقل اختلاف سرعت تقریباً 3.33 km/h می شود.

(1.8)

اگر فاصله دو جسم برابر باشد یعنی $t_{d1}=t_{d2}$ در نتیجه هر دو اکو فقط یک شیفت زمانی مشترک دارند اما چون سرعت ها متفاوت است فرکانس های داپلر و در نتیجه $f_{l1}=f_c+f_{d1}$ و $f_{l2}=f_c+f_{d2}$ متفاوت می شود و در حوزه فوریه معمولاً دو قله جدا دیده می شود و بنابراین می توان سرعت هر جسم را از روی مکان دو قله و رابطه $V=(f_{new}-f_c)/\beta$ به دست آورد. برای فاصله نیز چون تاخیر هر دو یکی است می توان t_d را از فاز هر کدام از مولفه ها استخراج کرد و چون هر دو باید به یک مقدار مشترک برسند این شرط حتی کمک می کند بین جواب های مدولار فاز مقدار درست انتخاب شود و در نهایت $R=td/\rho$ محاسبه می گردد. تنها شرط مهم این است که دو قله فرکانسی قابل تفکیک باشند یعنی اختلاف فرکانس ناشی از اختلاف سرعت از رزولوشن فرکانسی کوچک تر نباشد که طبق متن تمرین رزولوشن حدود $\delta f=1/T$ است و اگر اختلاف کمتر از آن شود دو هدف در طیف روی هم می افتند و جداسازی پارامترها ممکن نیست.

(1.9)

چون تعداد اهداف نامعلوم است ابتدا سیگنال دریافتی را به حوزه فرکانس می بریم و طیف را با FFT محاسبه می کنیم و مطابق راهنمای تمرین از روی اندازه طیف پیک ها را پیدا می کنیم و فاز متناظر هر پیک را نیز استخراج می کنیم سپس کف نویز را از بخش های کم انرژی طیف یا با هموارسازی برآورد می کنیم و یک آستانه انتخاب می کنیم تا هر پیکی که بالاتر از آستانه باشد یک هدف محسوب شود و بنابراین تعداد پیک های معتبر تعداد اهداف خواهد بود بعد برای هر پیک فرکانس را در صورت نیاز با zero padding یا اینترپولیشن محلی دقیق تر می کنیم و مانند بخش ۱ ۳ از f_{new} برای محاسبه f_d و سرعت و از فاز برای محاسبه t_d و فاصله

استفاده می کنیم و اگر دو هدف خیلی نزدیک باشند شرط تفکیک پذیری به رزولوشن فرکانسی $\delta f = 1/T$ وابسته است و در صورت همپوشانی پیک ها یا T را بزرگ تر می کنیم یا روش تکراری حذف مؤلفه را به کار می گیریم یعنی قوی ترین هدف را تخمین می زنیم سیگنال متناظر را بازسازی می کنیم از سیگنال اصلی کم می کنیم و این روند را تا زمانی ادامه می دهیم که انرژی باقی مانده از آستانه کمتر شود که در نهایت هم تعداد اهداف به صورت خودکار مشخص می شود و هم پارامترهای هر هدف جداگانه استخراج می گردد.

بخش دوم

(2.1)

در فایل run2.m

با فرض اینکه هر کلید یک موج سینوسی با فرکانس ثابت تولید می کند یک بردار زمان با نرخ نمونه برداری $f_s = 8000 \text{ Hz}$ ساخته شد و برای هر نت از رابطه $x(t) = \sin(2\pi f t)$ سیگنال مربوطه تولید گردید سپس برای جلوگیری از کلیک صوتی در ابتدا و انتهای هر نت یک محوشدگی کوتاه اعمال شد و بین نت های متوالی یک سکوت کوتاه حدود ۲۵ میلی ثانیه قرار گرفت در ادامه نت های خواسته شده مانند D و E و F# و G با مدت زمان های T و $T/2$ پشت سر هم چیده شدند تا ملودی ساخته شود و در نهایت با تابع `sound` پخش شد تا صحت تولید و ترتیب نت ها به صورت شنیداری بررسی گردد.

F:\Term7\Signal\run2.m

```
1 %2.1
2
3 clc; clear; close all;
4
5 fs = 8000;
6 tau = 0.025;
7
8 freq = containers.Map( ...
9     {'D','E','F#','G'}, ...
10     [293.6648, 329.6276, 369.9944, 391.9954] );
11
12 notes = { ...
13     'D' 'D' 'G' 'F#' 'D' 'D' 'E' 'E' 'D' 'F#' 'D' ...
14     'E' 'D' 'E' 'F#' 'E' 'D' 'E' 'E' 'D' 'F#' 'D' ...
15     'E' 'D' 'E' 'D' 'F#' 'E' 'D' 'E' 'D' 'F#' 'E' ...
16     'D' 'D' 'E' 'F#' 'E' 'F#' 'F#' 'E' 'F#' 'F#' 'D' ...
17 };
18
19 durs = [ ...
20     0.25 0.25 0.50 0.50 0.50 0.25 0.25 0.25 0.25 0.25 0.25 ...
21     0.50 0.50 0.50 0.50 0.50 0.25 0.25 0.25 0.25 0.25 0.25 ...
22     0.50 0.50 0.25 0.25 0.50 0.50 0.50 0.25 0.25 0.50 0.50 ...
23     0.25 0.25 0.50 0.25 0.25 0.50 0.25 0.25 0.50 0.50 0.50 ...
24 ];
25
26 gap = zeros(1, round(tau*fs));
27 fade = round(0.005*fs);
28
29 x = [];
30
31 for i = 1:numel(notes)
32     f0 = freq(notes{i});
33     dur = durs(i);
34
35     t = 0:1/fs:(dur-1/fs);
36     s = 0.8*sin(2*pi*f0*t);
37
38     if fade > 1 && numel(s) > 2*fade
39         w = ones(size(s));
40         w(1:fade) = linspace(0,1,fade);
41         w(end-fade+1:end) = linspace(1,0,fade);
42         s = s .* w;
43     end
44
45     x = [x s gap];
46 end
47
48 sound(x, fs);
49
```

(2.2

یک ملودی ساده (twinkle twinkle) با استفاده از فرکانس های استاندارد ۱۲ نت ساخته شد به این صورت که برای هر نت یک سیگنال سینوسی با دامنه مناسب و مدت زمان مشخص تولید گردید و بین نت ها یک سکوت کوتاه قرار داده شد سپس سیگنال نهایی به صورت یک بردار تک کاناله ذخیره شد و با دستور `audiowrite` قالب فایل wav با نرخ نمونه برداری ۸ کیلوهرتز نوشته شد و با استفاده از `audioinfo` مشخصات فایل خوانده شد تا مقدار `BitsPerSample` گزارش شود که با انتخاب ذخیره سازی ۱۶ بیتی هر نمونه با ۱۶ بیت در فایل نگهداری گردید و فایل تولید شده به عنوان ورودی بخش تشخیص نت ها نیز مورد استفاده قرار گرفت.

```
34 %2.2
35 fs = 8000;
36 T = 0.5;
37 tau = 0.025;
38
39 names = {'C','C#','D','D#','E','F','F#','G','G#','A','A#','B'};
40 freqs = [261.6256 277.1826 293.6648 311.1270 329.6276 349.2282 369.9944 391.9954 415.3047 440.0000 466.1638 493.8833];
41
42 getf = @(s) freqs(strcmp(names,s));
43
44 notes = {'C','C','G','G','A','A','G','F','F','E','E','D','D','C'};
45 durs = [T, T, T, T, T, T, T, T, T, T, T, T, T, T];
46
47 gap = zeros(1, round(tau*fs));
48 fade = round(0.005*fs);
49
50 x = [];
51
52 for i = 1:numel(notes)
53     f0 = getf(notes{i});
54     dur = durs(i);
55
56     tt = 0:1/fs:(dur-1/fs);
57     s = 0.8*sin(2*pi*f0*tt);
58
59     if fade > 1 && numel(s) > 2*fade
60         w = ones(size(s));
61         w(1:fade) = linspace(0,1,fade);
62         w(end-fade+1:end) = linspace(1,0,fade);
63         s = s .* w;
64     end
65
66     x = [x s gap];
67 end
68
69 x = x(:);
70 audiowrite('mysong.wav', x, fs, 'BitsPerSample', 16);
71
72 info = audioinfo('mysong.wav');
73 fprintf('BitsPerSample = %d\n', info.BitsPerSample);
74
75 sound(x, fs);
76
```

```
BitsPerSample = 16
>>
```

2.3

فرض می شود فایل موسیقی دقیقاً با مدل مسئله تولید شده است یعنی هر نت یک موج سینوسی تک فرکانسی از میان ۱۲ نت استاندارد است و بین نت ها سکوت کوتاه وجود دارد و مدت نگه داشتن کلید نیز معمولاً نزدیک به ۰٫۲۵ یا ۰٫۵ ثانیه است برای استخراج نت ها ابتدا سیگنال صوتی خوانده می شود و با محاسبه انرژی کوتاه مدت روی پنجره های کوچک نواحی فعال از سکوت جدا می گردد سپس با یک آستانه گذاری روی انرژی بازه های زمانی مربوط به هر نت پیدا می شود و با ادغام فاصله های خیلی کوتاه بین بازه ها مرزبندی نت ها دقیق تر می شود در این مرحله خروجی ما قطعه های مجزایی از سیگنال است که هر کدام باید متناظر با یک کلید فشرده شده باشد.

بعد از قطعه بندی برای تشخیص نوع نت از هر قطعه FFT گرفته می شود و فرکانس پیک طیف به عنوان فرکانس غالب همان قطعه استخراج می گردد و با مقایسه این فرکانس با جدول فرکانس های ۱۲ نت نزدیک ترین نت انتخاب می شود همزمان مدت نگه داشتن هر کلید از طول نمونه های قطعه تقسیم بر نرخ نمونه برداری به دست می آید و برای انطباق با فرض های تولید به نزدیک ترین مقدار از مجموعه ۰٫۲۵ و ۰٫۵ کوانتیزه می شود در پایان توالی نت ها به همراه زمان شروع و مدت هر نت گزارش می شود و برای اطمینان از صحت روش همین فرآیند روی فایل تولید شده در تمرین ۲۲ اجرا و نت های استخراج شده با ملودی اولیه مقایسه می گردد تا مشخص شود الگوریتم هم در تشخیص فرکانس نت و هم در برآورد مدت هر کلید عملکرد صحیح دارد.

Command Window						
idx	note	f_est_Hz	dur_s	dur_q	t_start_s	t_end_s
1	"C"	261.63	0.501	0.5	0	0.50087
2	"C"	261.63	0.50138	0.5	0.52463	1.0259
3	"G"	392	0.50125	0.5	1.0495	1.5506
4	"G"	392	0.50125	0.5	1.5745	2.0756
5	"A"	440	0.50175	0.5	2.0993	2.6009
6	"A"	440	0.50175	0.5	2.6242	3.1259
7	"G"	392	0.50125	0.5	3.1495	3.6506
8	"F"	349.23	0.50187	0.5	3.6741	4.1759
9	"F"	349.23	0.50187	0.5	4.1991	4.7009
10	"E"	329.63	0.5015	0.5	4.7241	5.2255
11	"E"	329.63	0.5015	0.5	5.2491	5.7505
12	"D"	293.66	0.50162	0.5	5.7744	6.2759
13	"D"	293.66	0.50162	0.5	6.2994	6.8009
14	"C"	261.63	0.50138	0.5	6.8246	7.3259

```

97 %2.3
98 clc; clear; close all;
99
100 wavFile = 'mysong.wav';
101
102 Tset = [0.25 0.5];
103
104 noteNames = {'C','C#','D','D#','E','F','F#','G','G#','A','A#','B'};
105 noteFreqs = [261.6256 277.1826 293.6648 311.1270 329.6276 349.2282 369.9944 391.9954 415.3047 440.0000 466.1638 493.8833];
106
107 [x, fs] = audioread(wavFile);
108 if size(x,2) > 1, x = mean(x,2); end
109 x = x(:);
110 mx = max(abs(x));
111 if mx > 0, x = x/mx; end
112
113 winEnv = max(1, round(0.010*fs));
114 pwr = x.^2;
115
116 if exist('movmean','file') == 2
117     env = movmean(pwr, winEnv);
118 else
119     env = conv(pwr, ones(winEnv,1)/winEnv, 'same');
120 end
121
122 thr = 0.10 * max(env);
123 act = env > thr;
124
125 d = diff([0; act; 0]);
126 sIdx = find(d == 1);
127 eIdx = find(d == -1) - 1;
128
129 minLen = round(0.10*fs);
130 keep = (eIdx - sIdx + 1) >= minLen;
131 sIdx = sIdx(keep);
132 eIdx = eIdx(keep);
133
134 mergeGap = round(0.015*fs);
135 s2 = []; e2 = [];
136 if ~isempty(sIdx)
137     cs = sIdx(1); ce = eIdx(1);
138     for k = 2:numel(sIdx)
139         if sIdx(k) - ce <= mergeGap
140             ce = eIdx(k);
141         else
142             s2 = [s2; cs];
143             e2 = [e2; ce];
144             cs = sIdx(k); ce = eIdx(k);
145         end
146     end
147     s2 = [s2; cs];
148     e2 = [e2; ce];
149 end
150
151 M = numel(s2);
152

```



```

155 durQ = zeros(M,1);
156 fHat = zeros(M,1);
157
158 for i = 1:M
159     seg = x(s2(i):e2(i));
160     durHat(i) = numel(seg)/fs;
161
162     Nfft = 2^nextpow2(max(4096, numel(seg)*8));
163     w = hannwin(numel(seg));
164     S = fft(seg.*w, Nfft);
165
166     mag = abs(S(1:floor(Nfft/2)+1));
167     mag(1) = 0;
168
169     [~,k0] = max(mag);
170     f = (0:floor(Nfft/2))*(fs/Nfft);
171
172     f0 = f(k0);
173     if k0 > 1 && k0 < numel(mag)
174         m1 = mag(k0-1); m0 = mag(k0); m2 = mag(k0+1);
175         den = (m1 - 2*m0 + m2);
176         if abs(den) > 1e-12
177             dk = 0.5*(m1 - m2)/den;
178             f0 = f0 + dk*(fs/Nfft);
179         end
180     end
181
182     fHat(i) = f0;
183     [~,ix] = min(abs(noteFreqs - f0));
184     notesHat(i) = string(noteNames{ix});
185
186     [~,iq] = min(abs(Tset - durHat(i)));
187     durQ(i) = Tset(iq);
188 end
189
190 tStart = (s2-1)/fs;
191 tEnd = (e2-1)/fs;
192
193 TBL = table((1:M)', notesHat, fHat, durHat, durQ, tStart, tEnd, ...
194     'VariableNames', {'idx','note','f_est_Hz','dur_s','dur_q','t_start_s','t_end_s'});
195 disp(TBL);
196
197 tt = (0:numel(x)-1)/fs;
198 figure; plot(tt, env); grid on; hold on;
199 plot(tt, thr*ones(size(tt)));
200 for i = 1:M
201     plot([tStart(i) tStart(i)], ylim);
202     plot([tEnd(i) tEnd(i)], ylim);
203 end
204 xlabel('t (s)'); ylabel('env');
205
206 function w = hannwin(L)
207     if L <= 1
208         w = ones(L,1);
209     else
210         n = (0:L-1).';
211         w = 0.5 - 0.5*cos(2*pi*n/(L-1));
212     end
213 end

```